DS-2002: Data Systems

Overview of Data Integration Processes

Prof. Jon Tupitza

University of Virginia

Modern Data Platform: Data Services Pipeline

INGEST

Data Orchestration & Monitoring

Data Integration

STORE

Unstructured Data Storage

Data Lake Storage

PREPARE

Data Cleansing, Transformation & Streaming

Databricks

SERVE

Data Modeling, Serving & Storing

Data Warehouse

PREDICT

Reporting, BI, Predictive Analytics & AI

Machine Learning

DELIVER

MLOps Integration: CI/CD Pipelines, Version Control, Monitoring, Test Automation, Infrastructure as Code, Containers, Microservices

DevOps

Data Integration

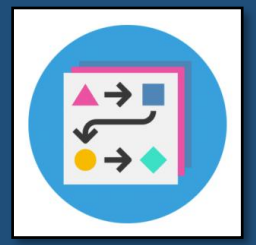
How to Approach Populating a Data Warehouse

Data Processing: Extract-Transform-Load (ETL)

Frequently, Data Must Be Moved from Sources to a Database and/or Data Lake

Extract

- This is the step where sensors wait for upstream data sources to land. Once available, we transport the data from their source locations to further transformations.



Transform

- The heart of any ETL job: apply business logic, perform actions such as filtering, grouping, and aggregation to translate raw data into analysis-ready datasets.

Load

- Load the processed data and transport to a final destination. Can now be consumed directly by end-users or treated as yet another upstream dependency.

Data Processing: ETL Process Design Patterns

Various Combinations of Activities May be Necessary to Satisfy Various Requirements

Extract (From Original Source Systems) → **Transform** (In the Pipeline) → **Load** (Serving Layer)

*The prototypical ETL process simply **extracts** from source, **transforms** in-flight, and then **loads** to a destination system.*

Extract (From Original Source Systems) → **Load** (Serving Layer) → **Transform** (In-Place on the Serving Layer)

*A variant **extracts** from source, and **loads** to a destination system. **Transformations** are applied in-place on the target.*

Extract (Original Source) → **Load** (Stage) → **Extract** (from Stage) → **Transform** (In the Pipeline) → **Load** (Serving Layer)

Source system limitations often require source data to be staged (loaded) locally before further processing can continue.

Data Processing: Batching versus Streaming

The rate at which incoming data must be integrated dictates how it must be processed

Data Motion	**At-Rest Data:** Data that has settled	**In-Motion Data:** Data where new events arrive at some continuous interval
Datasets	**Bounded Datasets:** Data of a known & finite size; having a start point and endpoint	**Unbounded Datasets:** Datasets wherein events are being continuously added
Data Processing Engines	**Batch Processing Engines:** Only capable of processing data after it has settled	**Stream Processing Engines:** Capable of processing data in-motion as it's arriving

Data Processing Paradigms: Latency Requirements

Latency & Response:
The speed at which clients require new insights determines the frequency at which new data must be processed

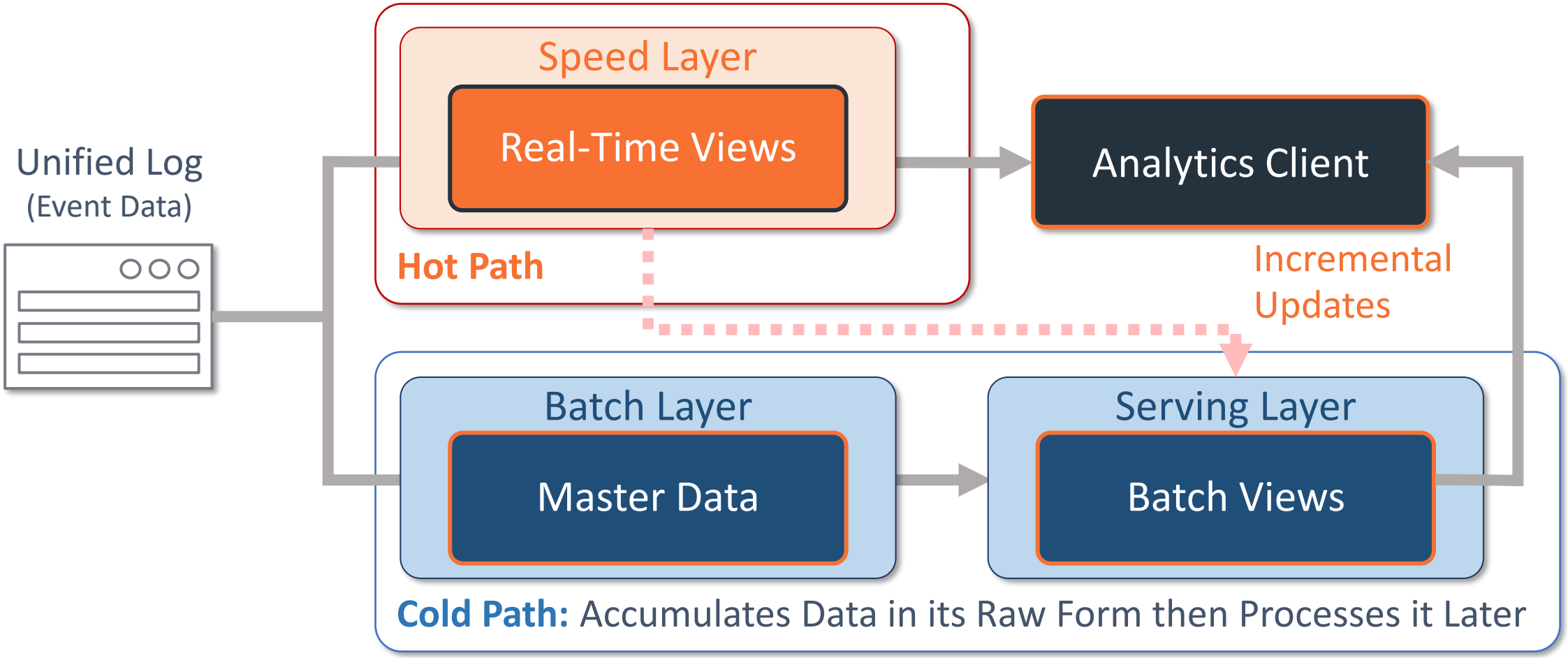
1. Batch
2. Continuous/Streaming
3. Real-time

10 ms → 100 ms → 1 sec → 1 min → 1 hour → 1 day

Low-Latency Real-Time	Real-Time	Micro-Batch	Batch
• Spark-less, highly-available prediction server	• Prediction server with Spark	• Structured Streaming	• Spark batch processing

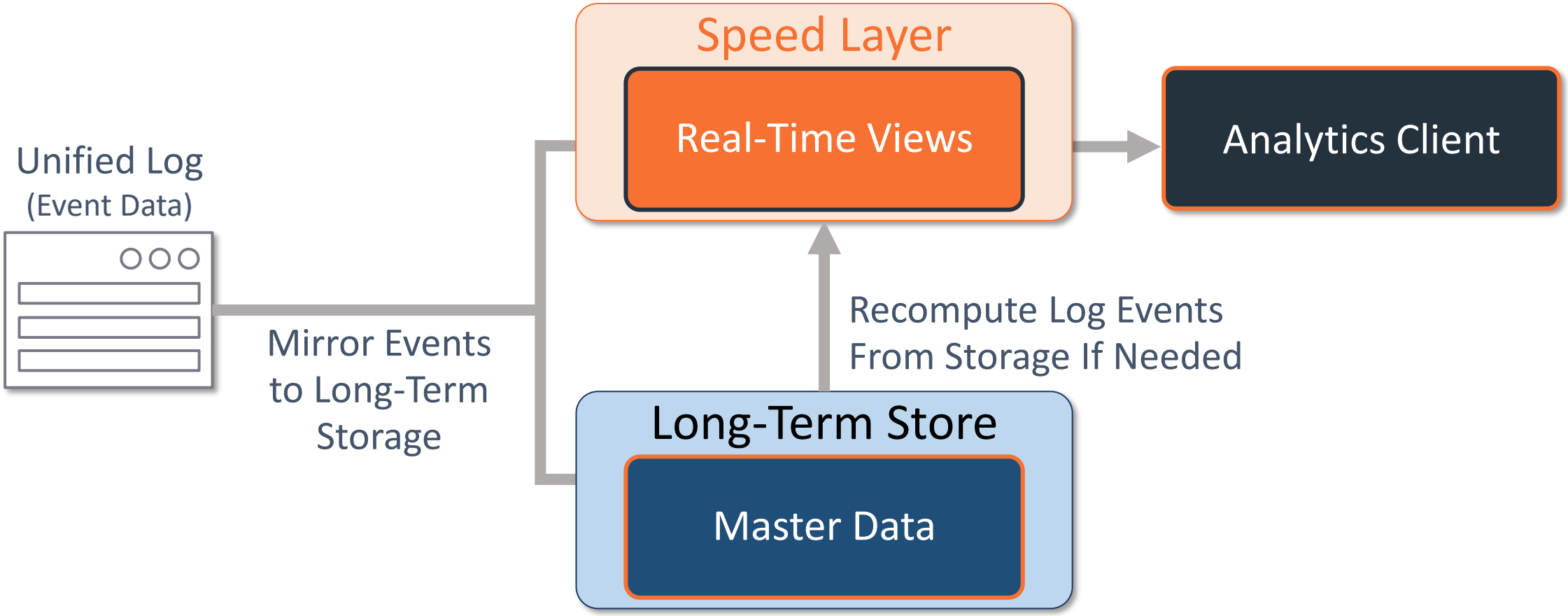
Data Processing Paradigms: Lambda Architecture

All Data Flows Through One of Two Paths: Hot or Cold



Data Processing Paradigms: Kappa Architecture

All Data Flows Through One of Two Paths: Hot or Cold



Paradigms: Data Storage and Retrieval

Schema on Write versus Schema on Read

Schema on Write: Stores data in an organized structure (schema)

Client or Process → Writes Data *to* → Schema

Client or Process ← Reads Data *from* ← Schema

Schema on Read: Applies schema only when read, data stored in its original format

Client or Process → Writes Data → *To* Storage

Client or Process ← Reads Data *into* Schema ← *From* Storage

Data Integration Patterns: Dimensional Data

Slowly Changing Dimension Update Strategies: Handling Variable Rates of Change

SCD Type 0

- Data in the Column Never Changes: Ever!
- Only for Static Reference Data

SCD Type 1

- No History is Maintained
- Existing Values are Overwritten by New Values
- **UPDATE**

SCD Type 2

- Historic Values are Maintained
- New Values are Written to a New Row
- *IsCurrent* Flag
- **INSERT**

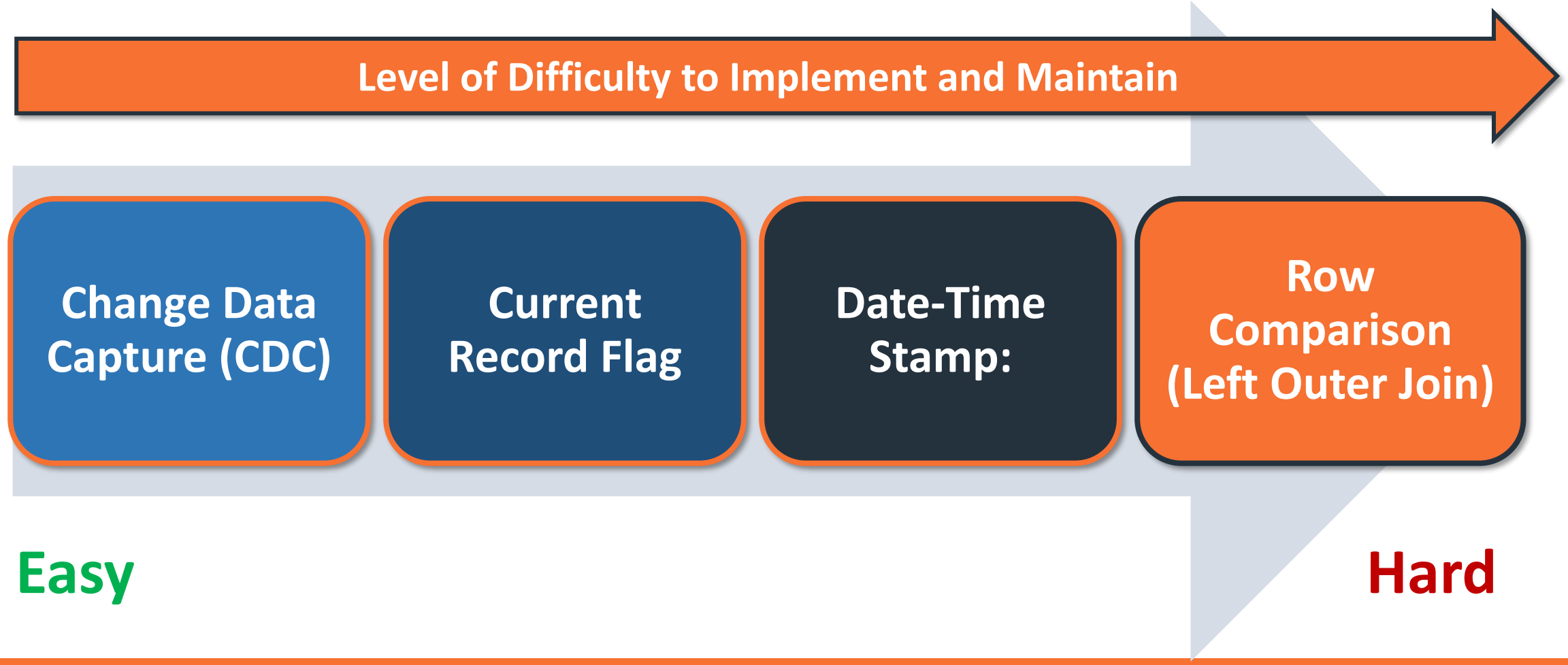
SCD Type 3

- A New Current Value Column is Created in the Existing Record
- Original Column is Also Retained

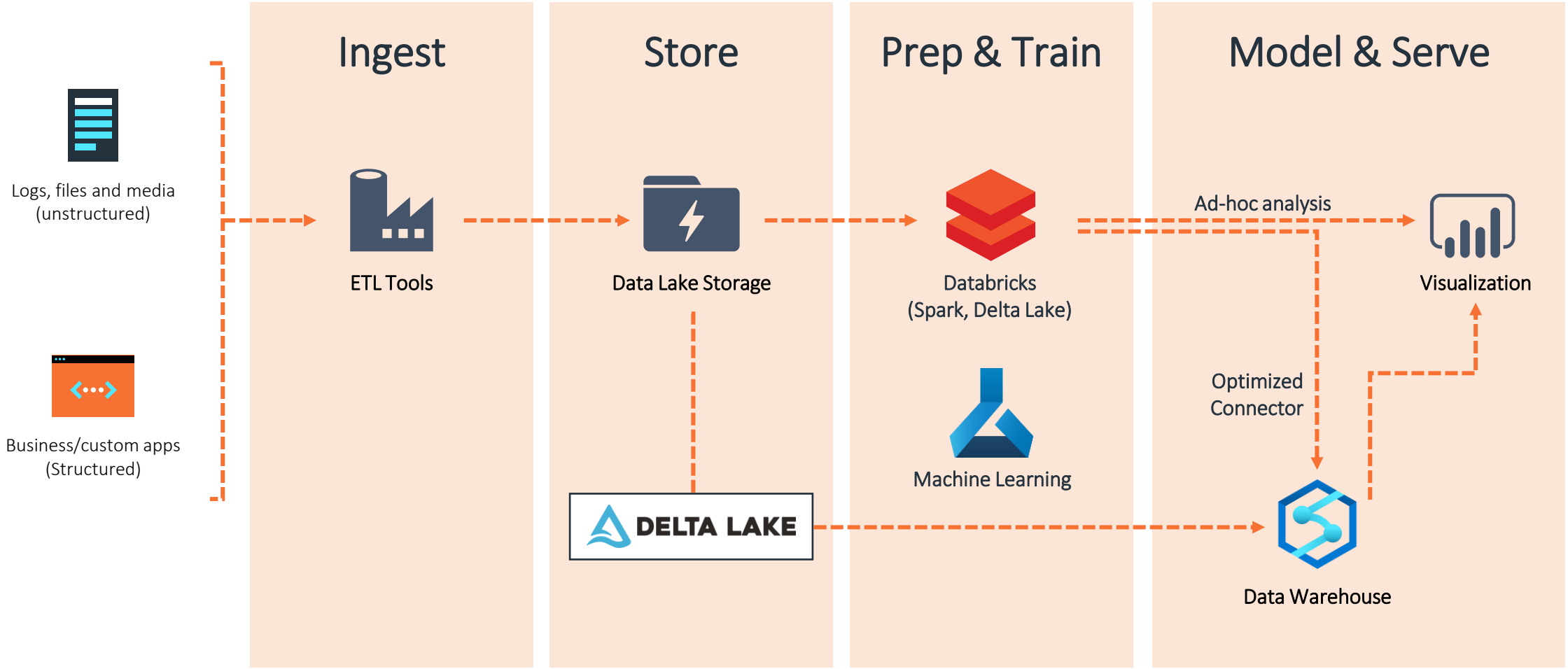
Easier to Implement and Maintain ← → **More Difficult to Implement and Maintain**

ETL Processing: Incremental Extraction

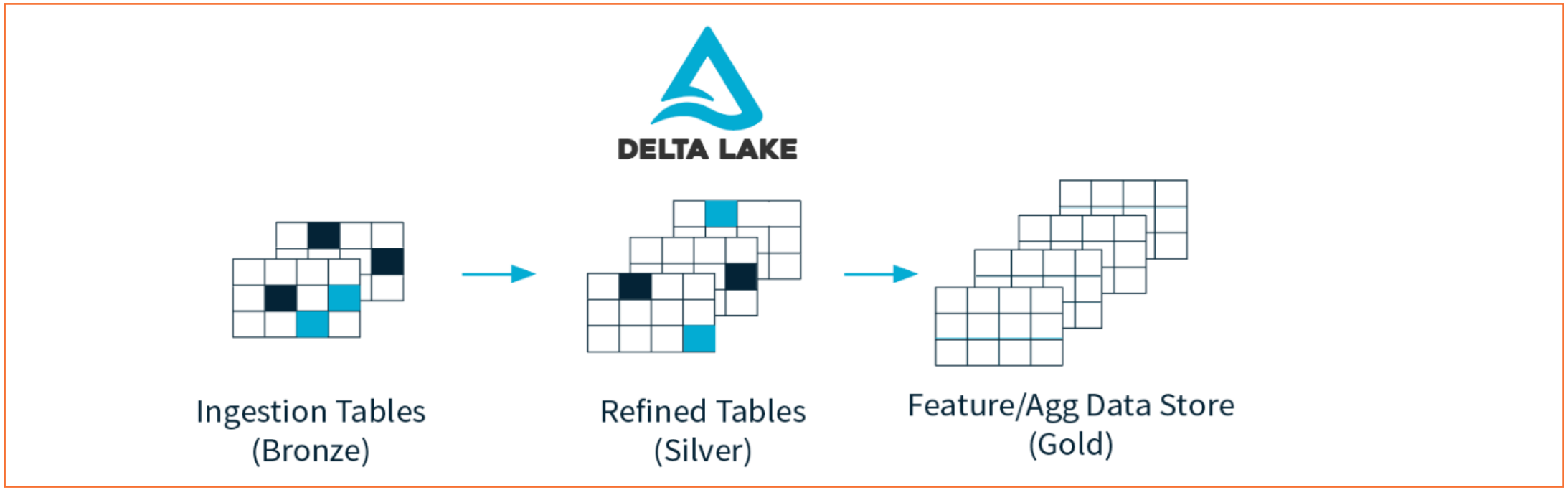
Techniques for Minimizing Data Movement: Extract Only the Changes



Data Engineering… for Data Science



Databricks: Delta Lake at Scale



ACID Transaction Guarantees

Atomic, Consistent, Isolated, Durable

Versioned Parquet Files

Delta transaction log keeps track of all operations

Efficient Upserts

MERGE, DELETE, UPDATE

Time Travel

Audit history, pipeline debugging, data reproducibility

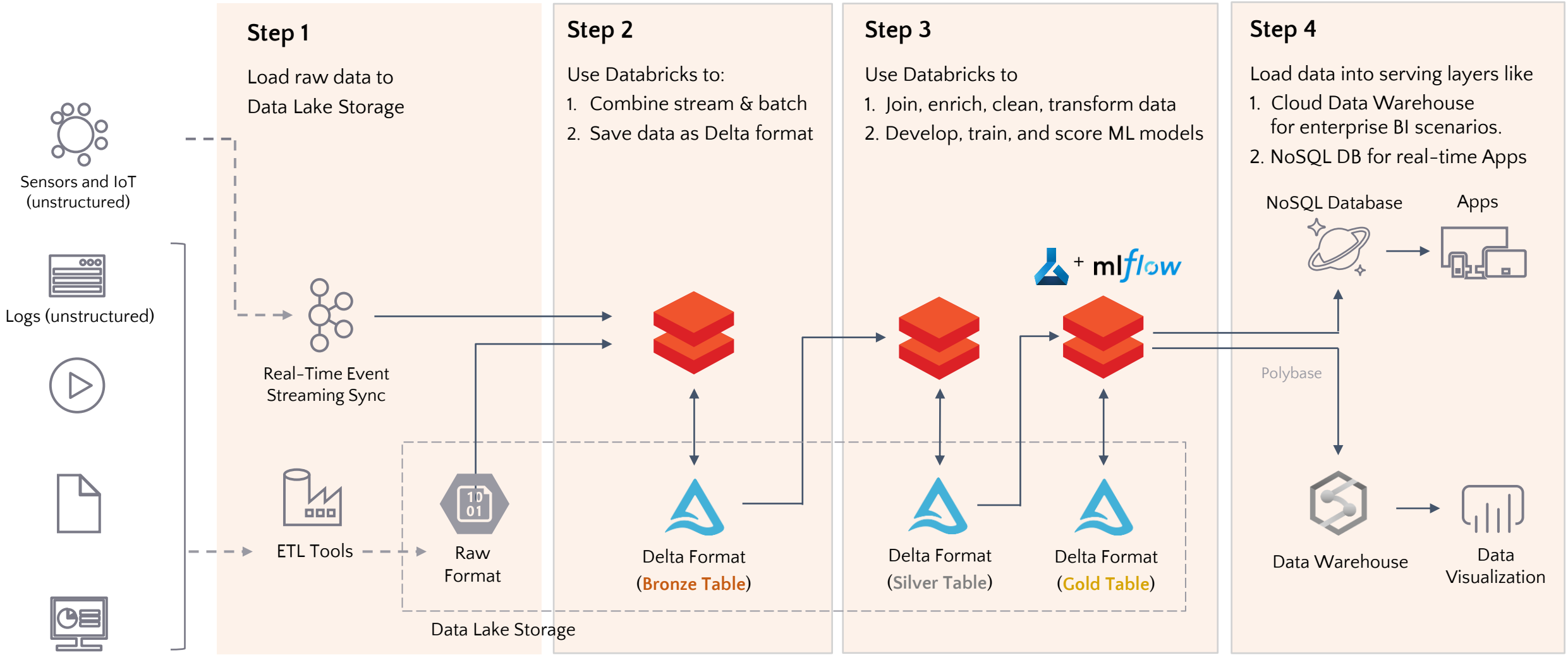
Small file compaction with no interrupt to availability

OPTIMIZE and VACUUM

Z-Order partitioning with up to 100x perf

New multidimensional partitioning enables data skipping

Design Pattern: Modern Data Warehousing



Q & A

An Overview of Data Warehouse Systems